Claude Code Docs

配置 › 自定义你的状态行

配置

自定义你的状态行

复制页面

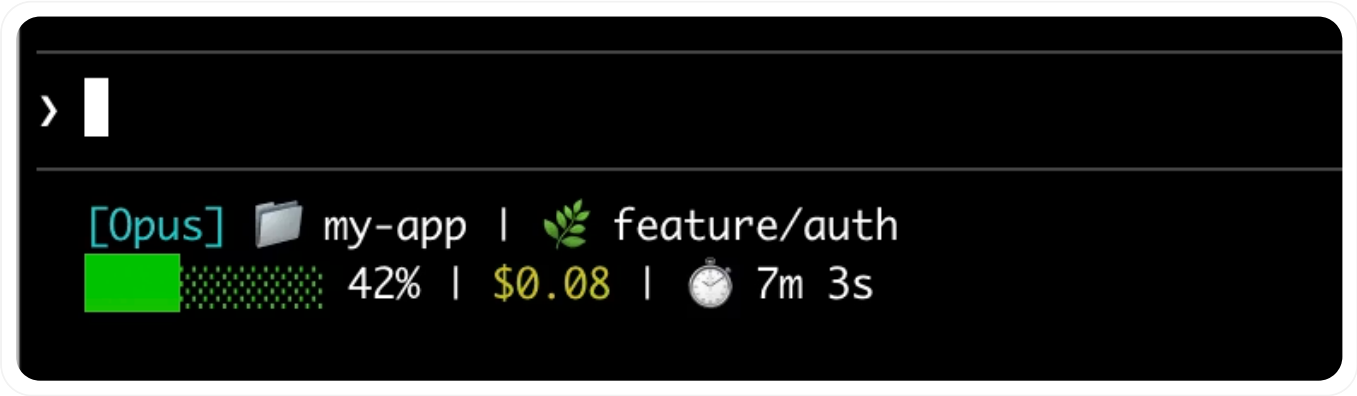
配置自定义状态栏以监控 Claude Code 中的上下文窗口使用情况、成本和 git 状态

状态行是 Claude Code 底部的可自定义栏，可以运行你配置的任何 shell 脚本。它通过 stdin 接收 JSON 会话数据，并显示你的脚本打印的任何内容，为你提供一个持久的、一目了然的上下文使用情况、成本、git 状态或任何其他你想跟踪的内容的视图。

状态行在以下情况下很有用：

- 你想在工作时监控上下文窗口使用情况
- 你需要跟踪会话成本
- 你在多个会话中工作，需要区分它们
- 你想让 git 分支和状态始终可见

这是一个**多行状态行**的示例，在第一行显示 git 信息，在第二行显示颜色编码的上下文栏。



本页面介绍了**设置基本状态行**、解释了**数据如何从 Claude Code 流向你的脚本**、列出了**你可以显示的所有字段**，并提供了**现成可用的示例**，用于 git 状态、成本跟踪和进度条等常见模式。

设置状态行

Claude Code Docs

›

使用/statusline命令

`/statusline` 命令接受描述你想显示内容的自然语言指令。Claude Code 在 `~/.claude/` 中生成脚本文件并自动更新你的设置：

```
/statusline show model name and context percentage with a pı
```

手动配置状态行

将 `statusLine` 字段添加到你的用户设置（`~/.claude/settings.json`，其中 `~` 是你的主目录）或 。将 `type` 设置为 `"command"`，并将 `command` 指向脚本路径或内联 shell 命令。有关创建脚本的完整演练，请参阅 。

```
{
  "statusLine": {
    "type": "command",
    "command": "~/.claude/statusline.sh",
    "padding": 2
  }
}
```

`command` 字段在 shell 中运行，所以你也可以使用内联命令而不是脚本文件。此示例使用 `jq` 解析 JSON 输入并显示模型名称和上下文百分比：

```
{
  "statusLine": {
    "type": "command",
    "command": "jq -r '\"[\\(.model.display_name)] \\(.context_window.used_perc
  }
}
```

Claude Code Docs

›

禁用状态行

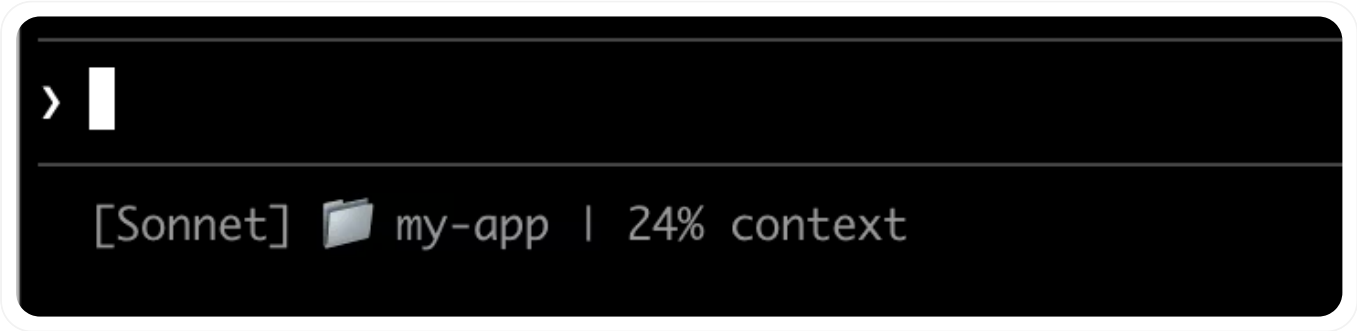
运行 `/statusline` 并要求它删除或清除你的状态行（例如，`/statusline delete`、`/statusline clear`、`/statusline remove it`）。你也可以手动从 settings.json 中删除 `statusLine` 字段。

逐步构建状态行

此演练展示了通过手动创建显示当前模型、工作目录和上下文窗口使用百分比的状态行来了解幕后发生的情况。

> 使用 `/statusline` 和你想要的内容描述会自动为你配置所有这些。

这些示例使用 Bash 脚本，在 macOS 和 Linux 上工作。在 Windows 上，你可以通过**WSL（Windows 子系统 for Linux）**运行 Bash 脚本或用 PowerShell 重写它们。



1. **创建一个读取 JSON 并打印输出的脚本**

   Claude Code 通过 stdin 向你的脚本发送 JSON 数据。此脚本使用 `jq`（一个你可能需要安装的命令行 JSON 解析器）来提取模型名称、目录和上下文百分比，然后打印格式化的行。

   将其保存到 `~/.claude/statusline.sh`（其中 `~` 是你的主目录，例如 macOS 上的 `/Users/username` 或 Linux 上的 `/home/username`）：

Claude Code Docs

```
# Extract fields using jq
MODEL=$(echo "$input" | jq -r '.model.display_name')
DIR=$(echo "$input" | jq -r '.workspace.current_dir')
# The "// 0" provides a fallback if the field is null
PCT=$(echo "$input" | jq -r '.context_window.used_percentage // 0' | cut

# Output the status line - ${DIR##*/} extracts just the folder name
echo "[$MODEL] 📁 ${DIR##*/} | ${PCT}% context"
```

2 **使其可执行**

将脚本标记为可执行，以便你的 shell 可以运行它：

```
chmod +x ~/.claude/statusline.sh
```

3 **添加到设置**

告诉 Claude Code 运行你的脚本作为状态行。将此配置添加到 `~/.claude/settings.json` ，它将 `type` 设置为 `"command"` （意思是"运行此 shell 命令"）并将 `command` 指向你的脚本：

```
{
  "statusLine": {
    "type": "command",
    "command": "~/.claude/statusline.sh"
  }
}
```

你的状态行出现在界面底部。设置会自动重新加载，但更改不会出现，直到你与 Claude Code 的下一次交互。

Claude Code Docs

›

何时更新

你的脚本在每条新的助手消息后、权限模式更改时或 vim 模式切换时运行。更新在 300ms 处进行防抖，这意味着快速更改会批处理在一起，你的脚本在事情稳定后运行一次。如果在你的脚本仍在运行时触发新的更新，则会取消正在进行的执行。如果你编辑你的脚本，更改不会出现，直到你与 Claude Code 的下一次交互触发更新。

你的脚本可以输出什么

多行：每个 `echo` 或 `print` 语句显示为单独的行。请参阅**多行示例**。

颜色：使用**ANSI 转义码**，如 `\033[32m` 表示绿色（终端必须支持它们）。请参阅**git 状态示例**。

链接：使用**OSC 8 转义序列**使文本可点击（macOS 上为 Cmd+click，Windows/Linux 上为 Ctrl+click）。需要支持超链接的终端，如 iTerm2、Kitty 或 WezTerm。请参阅**可点击链接示例**。

> 状态行在本地运行，不消耗 API 令牌。在某些 UI 交互期间，它会临时隐藏，包括自动完成建议、帮助菜单和权限提示。

可用数据

Claude Code 通过 stdin 向你的脚本发送以下 JSON 字段：

字段	描述
`model.id` , `model.display_name`	当前模型标识符和显示名称
`cwd` , `workspace.current_dir`	当前工作目录。两个字段包含相同的值；为了与 `workspace.project_dir` 保持一致，首选 `workspace.current_dir` 。
`workspace.project_dir`	启动 Claude Code 的目录，如果会话期间工作目录更改，可能与 `cwd` 不同
`cost.total_cost_usd`	总会话成本（美元）

Claude Code Docs

›

`cost.total_lines_added` ，`cost.total_lines_removed`	更改的代码行数
`context_window.total_input_tokens` ，`context_window.total_output_tokens`	整个会话中的累积令牌计数
`context_window.context_window_size`	最大上下文窗口大小（令牌）。默认为 200000，或对于具有扩展上下文的模型为 1000000。
`context_window.used_percentage`	预计算的已使用上下文窗口百分比
`context_window.remaining_percentage`	预计算的剩余上下文窗口百分比
`context_window.current_usage`	来自最后一次 API 调用的令牌计数，在**上下文窗口字段**中描述
`exceeds_200k_tokens`	最近一次 API 响应中的总令牌计数（输入、缓存和输出令牌合并）是否超过 200k。这是一个固定阈值，与实际上下文窗口大小无关。
`session_id`	唯一会话标识符
`transcript_path`	对话记录文件的路径
`version`	Claude Code 版本
`output_style.name`	当前输出样式的名称
`vim.mode`	启用**vim 模式**时的当前 vim 模式（ `NORMAL` 或 `INSERT` ）
`agent.name`	使用 `--agent` 标志或配置的代理设置运行时的代理名称

完整 JSON 架构

上下文窗口字段

`context_window` 对象提供了两种跟踪上下文使用情况的方式：

累积总计（ `total_input_tokens` ， `total_output_tokens` ）：整个会话中所有令牌的总和，用于跟踪总消耗

当前使用情况（ `current_usage` ）：来自最近一次 API 调用的令牌计数，使用此方法获得准确的上下文百分比，因为它反映了实际的上下文状态

Claude Code Docs

›

- `cache_creation_input_tokens` ：写入缓存的令牌
- `cache_read_input_tokens` ：从缓存读取的令牌

`used_percentage` 字段仅从输入令牌计算： `input_tokens + cache_creation_input_tokens + cache_read_input_tokens` 。它不包括 `output_tokens` 。

如果你从 `current_usage` 手动计算上下文百分比，使用相同的仅输入公式来匹配 `used_percentage` 。

`current_usage` 对象在会话中第一次 API 调用之前为 `null` 。

示例

这些示例展示了常见的状态行模式。要使用任何示例：

1. 将脚本保存到文件，如 `~/.claude/statusline.sh` （或 `.py` / `.js` ）
2. 使其可执行： `chmod +x ~/.claude/statusline.sh`
3. 将路径添加到你的**设置**

Bash 示例使用 `jq` 来解析 JSON。Python 和 Node.js 具有内置的 JSON 解析。

上下文窗口使用情况

显示当前模型和上下文窗口使用情况，带有可视进度条。每个脚本从 stdin 读取 JSON，提取 `used_percentage` 字段，并构建一个 10 字符的栏，其中填充的块（▓）代表使用情况：

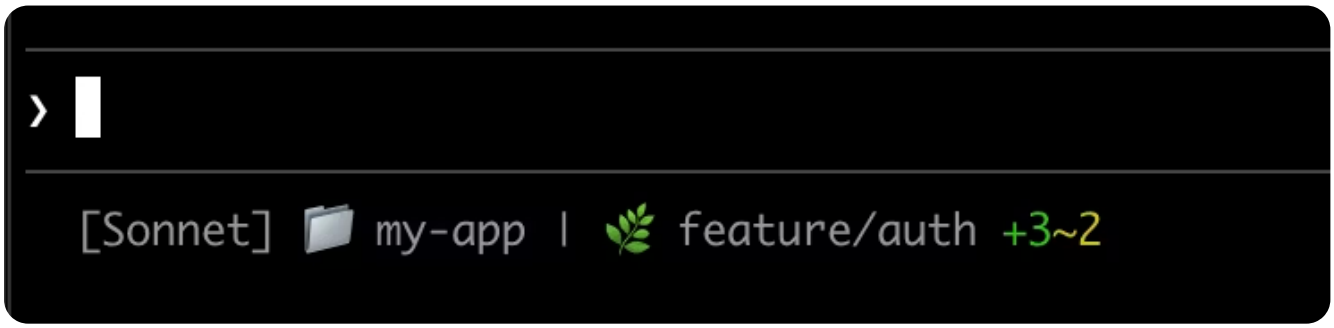
Claude Code Docs

›

```
# Extract fields with jq, "// 0" provides fallback for null
MODEL=$(echo "$input" | jq -r '.model.display_name')
PCT=$(echo "$input" | jq -r '.context_window.used_percentage // 0' | cut -d. -

# Build progress bar: printf creates spaces, tr replaces with blocks
BAR_WIDTH=10
FILLED=$((PCT * BAR_WIDTH / 100))
EMPTY=$((BAR_WIDTH - FILLED))
BAR=""
[ "$FILLED" -gt 0 ] && BAR=$(printf "%${FILLED}s" | tr ' ' '▓')
[ "$EMPTY" -gt 0 ] && BAR="${BAR}$(printf "%${EMPTY}s" | tr ' ' '░')"

echo "[$MODEL] $BAR $PCT%"
```

Git 状态与颜色

显示 git 分支，带有暂存和修改文件的颜色编码指示器。此脚本使用 表示终端颜色：`\033[32m` 是绿色，`\033[33m` 是黄色，`\033[0m` 重置为默认值。



每个脚本检查当前目录是否是 git 存储库，计算暂存和修改的文件，并显示颜色编码的指示器：

Claude Code Docs

›

```
MODEL=$(echo "$input" | jq -r '.model.display_name')
DIR=$(echo "$input" | jq -r '.workspace.current_dir')

GREEN='\033[32m'
YELLOW='\033[33m'
RESET='\033[0m'

if git rev-parse --git-dir > /dev/null 2>&1; then
    BRANCH=$(git branch --show-current 2>/dev/null)
    STAGED=$(git diff --cached --numstat 2>/dev/null | wc -l | tr -d ' ')
    MODIFIED=$(git diff --numstat 2>/dev/null | wc -l | tr -d ' ')

    GIT_STATUS=""
    [ "$STAGED" -gt 0 ] && GIT_STATUS="${GREEN}+${STAGED}${RESET}"
    [ "$MODIFIED" -gt 0 ] && GIT_STATUS="${GIT_STATUS}${YELLOW}~${MODIFIED}${R

    echo -e "[$MODEL] 📁 ${DIR##*/} | 🌿 $BRANCH $GIT_STATUS"
else
    echo "[$MODEL] 📁 ${DIR##*/}"
fi
```

成本和持续时间跟踪

跟踪你的会话的 API 成本和经过的时间。 `cost.total_cost_usd` 字段累积当前会话中所有 API 调用的成本。 `cost.total_duration_ms` 字段测量自会话开始以来的总经过时间，而 `cost.total_api_duration_ms` 仅跟踪等待 API 响应所花费的时间。

每个脚本将成本格式化为货币，并将毫秒转换为分钟和秒：

Claude Code Docs

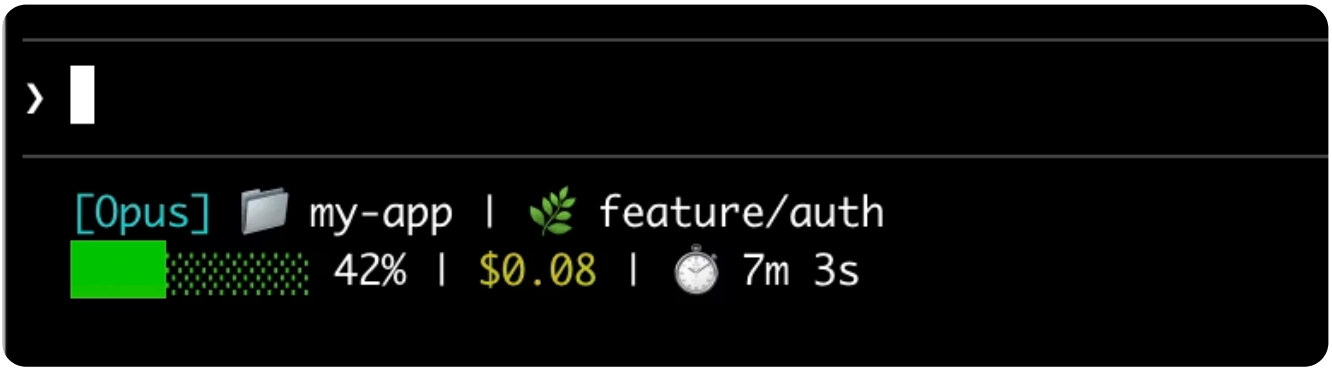
›

Bash Python Node.js

```bash
#!/bin/bash
input=$(cat)

MODEL=$(echo "$input" | jq -r '.model.display_name')
COST=$(echo "$input" | jq -r '.cost.total_cost_usd // 0')
DURATION_MS=$(echo "$input" | jq -r '.cost.total_duration_ms // 0')

COST_FMT=$(printf '$%.2f' "$COST")
DURATION_SEC=$((DURATION_MS / 1000))
MINS=$((DURATION_SEC / 60))
SECS=$((DURATION_SEC % 60))

echo "[$MODEL] 💰 $COST_FMT | ⏱️ ${MINS}m ${SECS}s"
```

显示多行

你的脚本可以输出多行来创建更丰富的显示。每个 `echo` 语句在状态区域中生成单独的行。



此示例结合了几种技术：基于阈值的颜色（70% 以下为绿色，70-89% 为黄色，90%+ 为红色）、进度条和 git 分支信息。每个 `print` 或 `echo` 语句创建单独的行：

Claude Code Docs

›

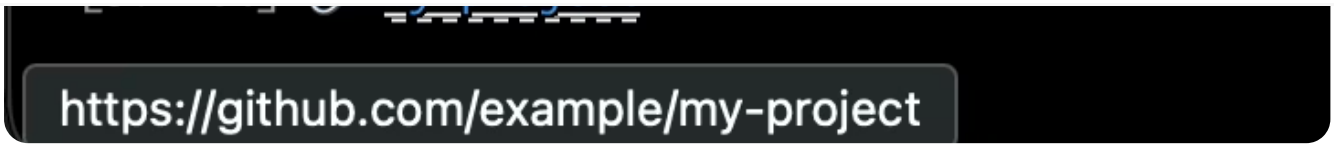
```
MODEL=$(echo "$input" | jq -r '.model.display_name')
DIR=$(echo "$input" | jq -r '.workspace.current_dir')
COST=$(echo "$input" | jq -r '.cost.total_cost_usd // 0')
PCT=$(echo "$input" | jq -r '.context_window.used_percentage // 0' | cut -d. -
DURATION_MS=$(echo "$input" | jq -r '.cost.total_duration_ms // 0')

CYAN='\033[36m'; GREEN='\033[32m'; YELLOW='\033[33m'; RED='\033[31m'; RESET='\

# Pick bar color based on context usage
if [ "$PCT" -ge 90 ]; then BAR_COLOR="$RED"
elif [ "$PCT" -ge 70 ]; then BAR_COLOR="$YELLOW"
else BAR_COLOR="$GREEN"; fi

FILLED=$((PCT / 10)); EMPTY=$((10 - FILLED))
BAR=$(printf "%${FILLED}s" | tr ' ' '▰')$(printf "%${EMPTY}s" | tr ' ' '▱')

MINS=$((DURATION_MS / 60000)); SECS=$(((DURATION_MS % 60000) / 1000))

BRANCH=""
git rev-parse --git-dir > /dev/null 2>&1 && BRANCH=" | 🌿 $(git branch --show-

echo -e "${CYAN}[$MODEL]${RESET} 📁 ${DIR##*/}$BRANCH"
COST_FMT=$(printf '$%.2f' "$COST")
echo -e "${BAR_COLOR}${BAR}${RESET} ${PCT}% | ${YELLOW}${COST_FMT}${RESET} | ⏱
```

可点击链接

此示例创建指向你的 GitHub 存储库的可点击链接。它读取 git 远程 URL，使用 `sed` 将 SSH 格式转换为 HTTPS，并将存储库名称包装在 OSC 8 转义码中。按住 Cmd（macOS）或 Ctrl（Windows/Linux）并单击以在浏览器中打开链接。

Claude Code Docs

›



每个脚本获取 git 远程 URL，将 SSH 格式转换为 HTTPS，并将存储库名称包装在 OSC 8 转义码中。Bash 版本使用 `printf '%b'` ，它比 `echo -e` 在不同 shell 中更可靠地解释反斜杠转义：

Bash Python Node.js

```
#!/bin/bash
input=$(cat)

MODEL=$(echo "$input" | jq -r '.model.display_name')

# Convert git SSH URL to HTTPS
REMOTE=$(git remote get-url origin 2>/dev/null | sed 's/git@github.com:/https:

if [ -n "$REMOTE" ]; then
    REPO_NAME=$(basename "$REMOTE")
    # OSC 8 format: \e]8;;URL\a then TEXT then \e]8;;\a
    # printf %b interprets escape sequences reliably across shells
    printf '%b' "[$MODEL] 🔗 \e]8;;${REMOTE}\a${REPO_NAME}\e]8;;\a\n"
else
    echo "[$MODEL]"
fi
```

缓存昂贵的操作

你的状态行脚本在活跃会话期间频繁运行。像 `git status` 或 `git diff` 这样的命令可能很慢，特别是在大型存储库中。此示例将 git 信息缓存到临时文件，并仅每 5 秒刷新一次。

为缓存文件使用稳定的固定文件名，如 `/tmp/statusline-git-cache` 。每个状态行调用作为新进程运行，所以基于进程的标识符（如 `$$` 、 `os.getpid()` 或 `process.pid` ）每次都产生不同的值，缓存永远不会被重用。

每个脚本在运行 git 命令之前检查缓存文件是否缺失或早于 5 秒：

Claude Code Docs

›

```
MODEL=$(echo "$input" | jq -r '.model.display_name')
DIR=$(echo "$input" | jq -r '.workspace.current_dir')

CACHE_FILE="/tmp/statusline-git-cache"
CACHE_MAX_AGE=5  # seconds

cache_is_stale() {
    [ ! -f "$CACHE_FILE" ] || \
    # stat -f %m is macOS, stat -c %Y is Linux
    [ $(($(date +%s) - $(stat -f %m "$CACHE_FILE" 2>/dev/null || stat -c %Y "$
}

if cache_is_stale; then
    if git rev-parse --git-dir > /dev/null 2>&1; then
        BRANCH=$(git branch --show-current 2>/dev/null)
        STAGED=$(git diff --cached --numstat 2>/dev/null | wc -l | tr -d ' ')
        MODIFIED=$(git diff --numstat 2>/dev/null | wc -l | tr -d ' ')
        echo "$BRANCH|$STAGED|$MODIFIED" > "$CACHE_FILE"
    else
        echo "||" > "$CACHE_FILE"
    fi
fi

IFS='|' read -r BRANCH STAGED MODIFIED < "$CACHE_FILE"

if [ -n "$BRANCH" ]; then
    echo "[$MODEL] 📁 ${DIR##*/} | 🌿 $BRANCH +$STAGED ~$MODIFIED"
else
    echo "[$MODEL] 📁 ${DIR##*/}"
fi
```

提示

- **使用模拟输入测试：** `echo '{"model":{"display_name":"Opus"},"context_window":{"used_percentage":25}}' | ./statusline.sh`

Claude Code Docs

›

社区项目如**ccstatusline**和**starship-claude**提供带有主题和其他功能的预构建配置。

故障排除

状态行未出现

- 验证你的脚本是可执行的： `chmod +x ~/.claude/statusline.sh`
- 检查你的脚本输出到 stdout，而不是 stderr
- 手动运行你的脚本以验证它产生输出
- 如果 `disableAllHooks` 在你的设置中设置为 `true` ，状态行也会被禁用。删除此设置或将其设置为 `false` 以重新启用。

状态行显示 `--` 或空值

- 字段在第一次 API 响应完成之前可能为 `null`
- 在你的脚本中使用回退处理 null 值，例如 jq 中的 `// 0`
- 如果多条消息后值仍为空，请重启 Claude Code

上下文百分比显示意外值

- 使用 `used_percentage` 获得准确的上下文状态，而不是累积总计
- `total_input_tokens` 和 `total_output_tokens` 在整个会话中是累积的，可能超过上下文窗口大小
- 上下文百分比可能与 `/context` 输出不同，因为每个计算的时间不同

OSC 8 链接不可点击

- 验证你的终端支持 OSC 8 超链接（iTerm2、Kitty、WezTerm）
- Terminal.app 不支持可点击链接
- SSH 和 tmux 会话可能根据配置剥离 OSC 序列
- 如果转义序列显示为文字文本，如 `\e]8;;` ，使用 `printf '%b'` 而不是 `echo -e` 以获得更可靠的转义处理

带有转义码的多行状态行比单行纯文本更容易出现渲染问题

脚本错误或挂起

以非零代码退出或不产生输出的脚本会导致状态行变为空白

慢速脚本会阻止状态行更新，直到它们完成。保持脚本快速以避免陈旧输出。

如果在慢速脚本运行时触发新的更新，正在进行的脚本会被取消

在配置之前使用模拟输入独立测试你的脚本

通知共享状态行行

系统通知，如 MCP 服务器错误、自动更新和令牌警告，显示在与你的状态行相同行的右侧

启用详细模式会向此区域添加令牌计数器

在窄终端上，这些通知可能会截断你的状态行输出

此页面对您有帮助吗？ 是 否

‹ 内存管理 自定义快捷键 ›

Claude Code Docs

Company

Anthropic

Careers

Economic Futures

Research

News

Help and security

Availability

Status

Support center

Claude Code Docs

›

Courses

MCP connectors

Customer stories

Engineering blog

Events

Powered by Claude

Service partners

Startups program

Privacy policy

Disclosure policy

Usage policy

Commercial terms

Consumer terms